High-Throughput Streaming Vector Similarity Joins on Multicore Processors

Anonymous Author

Abstract

Sliding-window vector similarity joins are becoming a core runtime primitive for large-model middleware, where fresh semantic relations must be materialized continuously from embedding streams and exposed as low-latency state for retrieval, agent memory, and online context fusion. Unlike static or batched ANN settings, this runtime must preserve window semantics while simultaneously handling high-rate insert, expire, route, and verify operations on shared-memory multicore hardware. The main systems difficulty is not a single index lookup, but the joint tension among high-parallelism recall, skew-aware routing cost, drift adaptation, and data-path efficiency.

We present *VSJoin*, implemented in *sageflow*, and its VSJoin execution path organized around four coordinated mechanisms: two-level centroid partitioning that decouples cross-worker routing from worker-local pruning; load-aware budgeted routing that adjusts fanout under skew rather than using fixed multicast; versioned online centroid split that adapts to drift without bulk migration by exploiting bounded sliding-window lifetime; and a zero-copy batched execution path that converts algorithmic locality into throughput on multicore processors. Together, these mechanisms target stable recall at high parallelism while bounding control-plane and data-movement overheads.

Our paper frames these mechanisms as implementation-aligned, testable propositions rather than unconditional claims. Under evaluated workloads, the VSJoin path is designed to recover recall at high parallelism, improve effective throughput, and stabilize skew-sensitive execution relative to prior execution paths and ablations. We explicitly report degradation boundaries, including aggressive skew, undersized routing budgets, and drift regimes where control lag becomes visible.

ACM Reference Format:
Anonymous Author. 2018. High-Throughput Streaming Vector Similarity Joins on Multicore Processors. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX).* ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Context: streaming vector joins in LLM middleware. High-dimensional embeddings are now first-class runtime data in retrieval-augmented generation [6], multi-step tool-using agents, and online memory systems. In these deployments, vector similarity joins are no longer offline analytics operators. They serve instead as a streaming middleware primitive: the system continuously ingests embedding streams, probes recent windows for neighbors, and materializes semantic state (relation edges, memory links, event-level context) that downstream components consume at microsecond latencies. Unlike batch or static ANN settings where index construction and query phases are cleanly separated [4, 8], this streaming variant requires joint handling of insert, expire, route, and verify operations on sliding-window state under strict latency and throughput targets on shared-memory multicore processors.

The fundamental challenge: granularity tensions in high-parallelism streaming vector joins. The systems challenge is not a single index lookup, but a set of coupled structural tensions that become visible when both parallelism and update churn are high. First, *routing granularity versus local pruning granularity*: any single-layer partitioning structure must simultaneously decide which worker owns a vector and how candidates are pruned locally. As parallelism increases, these two objectives diverge—routing needs fine granularity to preserve recall coverage, while local pruning wants compact index structure. A single layer cannot satisfy both. Second, *coverage under skew*: as vectors distribute unevenly across semantic regions, routing fanout must adapt to prevent hot regions from amplifying duplicates and skew without reverting to full broadcast. Third, *drift and ownership*: semantic distributions shift over time, making previously balanced routing regions hot. Traditional repartitioning approaches rely on synchronous state migration, which competes with the hot path. Fourth, *data-movement cost in the hot path*: even a better routing structure can fail to improve end-to-end throughput if vectors are deep-copied, reference-counted atomically, and processed record-by-record.

Related approaches and their limitations. Existing work falls into three categories, each addressing one or two of the above tensions but not all.

- **Static and batch ANN methods** (IVF [4], HNSW [8], learned indexes [5]) excel when data are stable and index construction can be amortized offline. They do not target continuous insert/expire/route dynamics, and their single-layer index design does not decompose routing and pruning separately. When applied to streams without modification, recall and latency become unstable under update churn.
- **Multicore stream joins on structured keys** (e.g., Dataflow, Flink, modern DBMSs [1]) excel at low-latency execution and fine-grained concurrency control via partitioning. Their partitioning logic depends on key orderability and stable key-locality; vectors lack both. Applying key-based partitioning to embeddings either over-simplifies (hash to a few coarse regions, losing recall) or creates per-dimension overhead without meaningful locality semantics.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

- **Distributed vector-stream systems** (spatial partitioning, hierarchical hashing [3]) reduce per-node compute via cross-machine communication. But inter-node communication, global coordination, and boundary replication overhead often dominate single-node multicore optimization. These designs do not directly map to shared-memory targets.

The fundamental gap is that no single category addresses the joint problem of high-parallelism routing structure, load-aware coverage under skew, drift-responsive control with bounded overhead, and data-path efficiency. Directly combining pieces from different categories (e.g., coarse IVF + fine-grained partitioning + global rebuild + deep copies) creates a system with conflicting constraints that fails under high parallelism or high churn.

Preliminary evidence of the problem. To motivate the design, we observe that common baseline approaches exhibit clear failure modes under the target operating regime.

- **Single-level centroid routing:** Using one centroid layer to both route and prune causes recall to collapse as parallelism P increases. A centroid set tuned for adequate routing coverage (say, $K = 4P$ regions) is too coarse for worker-local pruning, causing high candidate fanout and poor recall-cost trade-offs. Conversely, tuning K for local pruning (say, $K = \sqrt{N}$) undercovers cross-worker routing, missing boundary matches.
- **Fixed unicast partitioning:** Assigning each record to a single worker via hash or range partitioning reduces synchronization but risks missing near-boundary true pairs, especially as P grows. Strict unicast on uniform distributions works; on skewed distributions, it concentrates traffic on a few workers.
- **Shared mutable state with locks:** Maintaining a single global index shared among workers via read-write locks reduces memory overhead but creates high contention on insert/query paths. Under high-rate concurrent inserts and probes, lock-wait times dominate the critical path, eroding throughput gains from better indexing.

These observations motivate our redesign. Rather than trying to patch a single structure or add locks, we decompose the problem into four coordinated mechanisms that each target one aspect of the tension.

Our approach: four coordinated mechanisms. *VSJoin* in *sageflow* implements a streaming vector-join execution path organized around four complementary design decisions:

- **Two-level centroid partitioning** separates routing granularity (coarse) from local pruning (fine), allowing routing to follow parallelism while pruning follows worker-local state size.
- **Load-aware budgeted routing** adapts per-centroid fanout based on coarse-region load, using power-of-two-choices tie breaking to prevent skew amplification without full broadcast.
- **Versioned online split** detects hot semantic regions and splits them atomically into new routing-table versions, adapting to drift while avoiding bulk migration (exploiting the bounded lifetime of sliding-window state).
- **Zero-copy batched execution** keeps the data path pointer-based and micro-batched, converting routing locality into cache- and SIMD-friendly execution instead of losing gains to copying and scalar overhead.

These mechanisms are complementary: removing any one reverts toward a known failure mode. The design explicitly treats routing, split, and batching as scoped system mechanisms (not semantic guarantees) whose value must be measured through recall, effective throughput, latency, and load-balance evidence.

Testable contributions. In summary, this paper makes the following **testable contributions**:

- **Proposition P1 (Two-level partitioning for high-parallelism recall).** We evaluate whether separating coarse routing from fine local pruning recovers recall as parallelism increases, relative to single-level partitioning strategies that must use one structure for both goals. Evidence: scaling curves and ablation removing the fine layer.
- **Proposition P2 (Load-aware budgeted routing under skew).** We evaluate whether adaptive fanout with load-aware tie breaking provides a better recall–cost–imbalance trade-off than fixed-budget routing extremes under skewed workloads. Evidence: comparison with unicast, fixed multicast, and broadcast routing across skewed distributions; load-imbalance metrics.
- **Proposition P3 (Versioned online split for drift without bulk migration).** We evaluate whether versioned routing-table publication and hot-centroid split can adapt to semantic drift while keeping control overhead bounded and avoiding full historical state migration. Evidence: drift-trace experiments with recovery-time and transition-recall measurements; split overhead breakdown.
- **Proposition P4 (Zero-copy batched execution for effective throughput).** We evaluate whether pointer-based multicast, epoch-managed lifetime, and micro-batched distance evaluation convert structural routing gains into end-to-end throughput without unacceptable p99 latency inflation. Evidence: throughput comparison with copy-heavy variants; latency and breakdown analysis.
- **Evidence scope and degradation boundaries.** We report implementation-aligned evidence with explicit boundaries, including degraded regimes under aggressive skew, undersized routing budgets, split-transition lag, and over-batching, and limit conclusions to tested hardware, runtime settings, and workloads.

Organization. The rest of this paper is organized as follows. Section 2 formalizes the problem, discusses the three coupled structural tensions, and presents preliminary baseline evidence motivating the redesign. Section 3 presents the core architecture and the four mechanisms in *Goal* → *Mechanism* → *Trade-off* form so that each mechanism directly maps to one proposition. Section 4 aligns code-level semantics with the architecture. Section 5 describes experimental methodology and evaluates P1–P4 with end-to-end, scaling, stress, and ablation evidence. Section 6 contextualizes our work within streaming systems, vector indexing, and multicore optimization. Section 7 concludes and discusses future directions.

2 Problem Statement and Motivation

We study streaming vector similarity joins over two unbounded streams L and R. Each tuple is represented as $(uid, ts, \mathbf{v})$, where $\mathbf{v} \in \mathbb{R}^d$ is an embedding vector and ts is event time. For a given window size W and similarity threshold θ, a join result is an ordered pair (l, r) such that

$$|l.ts - r.ts| \le W, \qquad \mathrm{sim}(l, r) \ge \theta.$$

The system continuously outputs valid pairs as tuples arrive and old tuples expire.

This operator is important in large-model middleware because the output pairs act as rapidly materialized semantic state. Instead of waiting for offline indexing or periodic batch refresh, downstream middleware components can directly consume fresh join results as evidence links, retrieval hints, memory edges, or event-level semantic associations. The operator is therefore judged not only by approximate-search quality, but also by whether it can sustain streaming state materialization under window semantics.

For an arrival x at time t, define the opposite-side active set

$$\mathcal{A}_{\bar{s}}(t) = \{y \mid y \in \bar{s},\ t - W \le y.ts \le t + W\}.$$

Candidate generation returns $C(x) \subseteq \mathcal{A}_{\bar{s}}(t)$, and verification outputs

$$\mathcal{J}(x) = \{(x, y) \mid y \in C(x),\ \mathrm{sim}(x, y) \ge \theta\}.$$

Over a run horizon T, let $\mathcal{J}^*$ be exact join results and $\widehat{\mathcal{J}}$ be system output. We measure

$$\text{Recall} = \frac{|\widehat{\mathcal{J}} \cap \mathcal{J}^*|}{|\mathcal{J}^*|}, \qquad \text{Precision} = \frac{|\widehat{\mathcal{J}} \cap \mathcal{J}^*|}{|\widehat{\mathcal{J}}|}.$$

The system objective is to maximize throughput and minimize tail latency while maintaining high recall/precision under bounded memory. In the target multicore setting, we also care about scaling efficiency and load imbalance because poor parallel scaling directly reduces the usefulness of streaming state materialization.

Three coupled tensions in high-parallelism streaming vector joins. The redesign of VSJoin is driven by three structural tensions that become visible once parallelism and update churn are both high.

- **Routing granularity versus local pruning granularity.** A single centroid layer must simultaneously decide which worker should own a vector and how candidates should be pruned locally. As parallelism grows, these two goals diverge: coarse routing wants more partitions to preserve recall, while local pruning wants compact per-worker structure. Using one layer for both typically causes recall collapse or excessive fanout.
- **Coverage versus skew cost.** Increasing fanout improves boundary coverage, but fixed multicast budgets ignore load skew and semantic hotspots. Full broadcast is too expensive; fixed unicast under-covers near-boundary true pairs; fixed multicast performs poorly once coarse regions become unevenly loaded.
- **Drift adaptation versus state movement.** In sliding windows, the semantic distribution can drift over time, making previously balanced routing regions hot. Traditional repartitioning approaches often rely on synchronous state migration or global rebuilds. Under continuous streams, these control actions compete with the hot path and can erase throughput gains.

Preliminary observations: failure modes of baseline approaches. To motivate the design, we conducted a series of measurements on common baseline strategies applied to the target regime (high parallelism $P \in \{4, 8, 16, 32\}$, continuous churn, moderate to high skew). The results clearly exhibit the structural gaps described above.

- **Single-layer centroid routing** (tuning one K-centroid set for both routing and pruning): Under $P = 16$, a centroid set sized for adequate routing ($K = 64$) results in high local probe cost and recall ≈ 0.68. Conversely, tuning K for efficient local pruning ($K = 128$) under-covers cross-worker routing, causing recall to drop to ≈ 0.57 due to missed boundary matches. The fundamental problem is that one layer cannot satisfy both goals.
- **Fixed-fanout unicast partitioning** (round-robin partition assignment, no multicast): Under uniform data, this maintains reasonable recall (≈ 0.82). Under Zipf(1.5) skewed distribution, however, hot partitions experience 2–3× higher load, causing tail latency to spike to > 3 ms at $P = 16$, while cold partitions remain idle. Strict unicast is too rigid.
- **Shared mutable global index with read-write locks**: Maintaining a single global centroid index shared across workers via locks eliminates memory overhead but creates severe contention on insert and query paths. Lock-wait time dominates, consuming 30–40% of the critical path even at moderate rates (10 K records/s). This rules out shared-state approaches for the target throughput.

These observations directly motivate our design: we need routing separation (avoiding single-layer dilemma), load-aware coverage (handling skew without unicast brittleness), drift responsiveness (avoiding bulk migration), and lock-free hot paths (eliminating synchronization overhead).

Correctness anchor versus system heuristics. To avoid conflating output semantics with optimization heuristics, we separate VSJoin into a correctness anchor and a set of system mechanisms. The correctness anchor determines whether an emitted pair is valid: window-time validity, similarity-threshold verification, and duplicate-safe output semantics. The routing, split, and batching mechanisms decide how candidates are discovered and how work is distributed. They affect recall, cost, and stability, but they do not redefine correctness. This separation is important for the paper because it lets us treat routing and split decisions as scoped system propositions rather than as semantic guarantees.

Why the old structure is insufficient. Our previous VSJoin path relied on a single-layer centroid structure together with periodic global rebuild and fixed fanout. That structure is no longer adequate for the target operating point. First, fixed partition counts and single-layer centroid routing do not scale recall with parallelism. Second, fixed fanout ignores centroid-level skew, so traffic concentration inflates both duplication and imbalance. Third, control actions centered on rebuild or remapping remain too coarse when semantic hotspots move quickly. Finally, even when routing improves,

deep copies and shared ownership in the data path can hide the algorithmic gains behind memory traffic and synchronization.

Why existing approaches are insufficient. Existing work can be roughly grouped into three categories, and each category misses at least one requirement of our target setting.

- **Multicore stream joins for structured keys.** These systems are strong on low-latency stream execution and concurrency control, but their partitioning logic depends on key orderability and key-based locality. In vector joins, embeddings do not provide a strict total order usable as a stable key, so direct key-partition transfer is ineffective for similarity-aware routing.
- **Static or batch vector similarity joins/search.** ANN and index-centric methods excel when data are mostly stable [4, 8], but streaming windows require continuous insert+expire and routing changes under update churn. Their typical design assumptions do not address the interaction among routing granularity, skew, and bounded-lifetime state.
- **Distributed vector-stream solutions.** Spatial partitioning and hash-based routing can reduce local compute, but cross-node communication, global coordination, and boundary replication overheads often dominate. These designs do not directly map to single-node shared-memory multicore optimization targets.

Therefore, directly transplanting any single category rarely satisfies all three goals simultaneously: (i) sustained multicore throughput, (ii) low end-to-end latency, and (iii) stable join quality under high-rate sliding-window updates.

Implication for VSJoin design. These gaps motivate a unified runtime where routing structure, local pruning, drift adaptation, and data-path lifetime are co-designed. In the current redesign, this leads to four mechanism-level questions that match the introduction propositions:

- **P1 / two-level partitioning:** how to separate cross-worker routing from worker-local pruning so recall does not collapse at high parallelism.
- **P2 / load-aware budgeted routing:** how to improve coverage under skew without paying broadcast-like duplication cost.
- **P3 / versioned online split:** how to adapt to hot or drifting semantic regions without bulk migration.
- **P4 / zero-copy batched execution:** how to turn structural routing improvements into effective throughput instead of losing them to copying, reference counting, and per-record scalar execution.

Accordingly, the next section presents VSJoin not as one monolithic index, but as a coordinated architecture with a versioned control plane and a pointer-based batched data plane. The paper evaluates these mechanisms as scoped system claims with explicit degradation boundaries under evaluated workloads.

The next section presents this architecture and its method-specific execution paths, including baseline routes and the VSJoin-oriented path.

[Figure Placeholder: Zero-Copy Batched Execution Path]

Visual: Data path comparison showing:

- Copy-heavy path: record-by-record copies, atomic ref-count increments
- Zero-copy path: pointer multicast, epoch lifetime guard, batch distance kernels
- Memory traffic comparison
- Instruction count and IPC (instructions per cycle) improvement
- Latency p50/p99 distribution under different batch sizes

[Figure Placeholder: Versioned Routing Table with Online Split]

Visual: Timeline showing:

- Time axis with arrival stream
- Load signal per coarse region
- Split trigger detection (hot region threshold crossing)
- RCU atomic table swap (v1 → v2)
- Old vs new routing for records in transition
- Natural expiration without migration

[Figure Placeholder: Two-Level Centroid Structure]

Visual: Two-layer diagram showing:

- Global coarse centroids ($K_c = 4P$ regions, e.g., 64 for $P = 16$)
- Routing decision per coarse centroid
- Worker-local fine centroids ($K_f \approx \sqrt{N_{\text{local}}/K_c}$)
- Routing coverage vs local pruning tradeoff illustration

[Figure Placeholder: Load-Aware Budget Allocation]

Visual: Load vs fanout budget curve showing:

- X-axis: coarse centroid load $\rho(c)$ normalized to mean
- Y-axis: routing budget allocation budget(c)
- Power-mean scaling curve: budget$(c) = B_{\text{base}}/\sqrt{\rho(c)}$
- Unicast, fixed multicast, adaptive, and broadcast extremes

3 Core Join Architecture and Execution Paths

Section 2 identified four coupled design questions behind the current VSJoin redesign. This section presents the resulting architecture in *Goal* → *Mechanism* → *Trade-off* form so that implementation and experiments can align directly with propositions P1–P4 in Section 1.

3.1 Execution Contract and Design Goals

For an arriving record x from side $s \in \{L, R\}$, VSJoin follows an eager pipeline: micro-batch admission, coarse routing, owner-state update, worker-local candidate retrieval, verification, and emission.

A pair (x, y) is emitted only if

$$|x.ts - y.ts| \leq W \quad \wedge \quad \text{sim}(x, y) \geq \theta.$$

Hence, correctness remains anchored in verification, while routing, split, and batching policies determine coverage and cost.

We organize the design around four goals:

- **G1 (P1):** recover recall at high parallelism by separating routing granularity from local pruning granularity.
- **G2 (P2):** improve boundary coverage under skew without paying fixed broadcast-like routing cost.
- **G3 (P3):** adapt to hot or drifting semantic regions through a versioned control plane with bounded overhead.
- **G4 (P4):** keep the hot path copy-light and batch-friendly so structural gains translate into throughput.

3.2 Mechanism I: Two-Level Centroid Partitioning (for G1)

Design goal. High-parallelism recall preservation requires a structural redesign at the partitioning level. A single centroid layer is fundamentally insufficient because routing and local pruning optimize conflicting objectives. To see why, suppose we fix a centroid count K and use it to both route vectors across workers and prune candidates locally.

- **Tuning K for routing:** To avoid collapsing many semantic neighborhoods onto one worker under $P = 16$ or $P = 32$, routing needs $K \approx 4P$ regions (e.g., $K = 128$). But such a coarse granularity per worker leads to large per-worker candidate sets and poor pruning efficiency—the system either overshoots fanout or undershoots recall.
- **Tuning K for local pruning:** To keep per-worker probe costs low, K should be smaller and more adaptive to local state size (e.g., $K = \sqrt{N_{\text{local}}}$ ranges from 20 to 100). But a coarse global K sized this way leaves routing coverage severely under-provisioned: many boundary vectors end up routed to a single worker and are never probed by its neighbors.

This inherent conflict is not a tuning problem; it is a structural one. A single layer cannot simultaneously satisfy a routing objective (preserve coverage across parallelism) and a pruning objective (minimize per-worker probe cost). The resolution is to separate routing and pruning into two independent layers.

Mechanism sketch. VSJoin decouples routing and pruning through a two-layer centroid hierarchy:

- **Coarse centroids** (count K_c, typically $\approx 4P$) define the routing partition structure. Each coarse region determines a primary owner and a bounded replica set. Coarse centroids scale with parallelism so routing granularity does not become a bottleneck.
- **Fine centroids** (count K_f, typically $\approx \sqrt{N_{\text{local}}/K_c}$) are maintained independently by each worker for its owned coarse regions. Fine centroids are tuned for local candidate pruning and scale with worker-local state size, not with global parallelism.

For a query record x, VSJoin's candidate discovery proceeds in two stages. First, probe coarse centroids to find the set of routing targets (primary worker plus bounded replicas):

$$\mathcal{R}_{\text{coarse}}(x) = \text{TopNprobe}(\text{coarse\_dist}(x), K_c, n^c_{\text{probe}}).$$

Second, within each target worker's owned coarse region, probe fine centroids to retrieve candidates:

$$C(x) = \text{Dedup}\Big(\bigcup_{c \in \mathcal{R}_{\text{coarse}}(x)} C_{\text{fine}}(x, c) \Big),$$

where fine centroids inside coarse region c are probed with a worker-local threshold. This design eliminates the mutual constraint: routing can follow parallelism, and pruning can follow state locality.

When the separation works. The power of this design emerges from two observations. First, a coarse region sized to 10–100 dimensions or suitable centroid count rarely maps all true neighbors onto a single worker; reasonable routing coverage is achievable. Second, within a worker's local state, an independent fine-centroid index can be trained and tuned to minimize local probe cost without being forced to compromise at the global level. Routing recall is decoupled from pruning efficiency.

Trade-off and scope. Two-level partitioning introduces control and metadata complexity. Suboptimal cardinality choices still hurt: too few coarse regions under-cover routing; too many inflate route cost; poor fine-centroid design still wastes local probes. The claim is not that one parameter setting is universally optimal, but that the separation allows routing and pruning to be tuned independently and thus recovers recall at high parallelism relative to single-layer constraints.

Proposition linkage (P1). This mechanism yields a testable claim: separating coarse routing from worker-local fine pruning recovers recall as parallelism rises, relative to single-level routing/pruning structures constrained to a single centroid cardinality.

3.3 Mechanism II: Load-Aware Budgeted Routing (for G2)

Design goal. Even with two-level partitioning, routing coverage and parallelism create a new tension when data are skewed. Boundary coverage requires multicast (routing records to multiple workers for verification), but uniform fixed-budget multicast ignores semantic hotspots and amplifies skew.

Consider a Zipf(1.5) distribution across coarse regions. If the top-hottest coarse region is 5–10× busier than the median, a fixed unicast budget throws away true matches (low recall). A fixed multicast budget spreads the same number of routes equally, wasting routes on quiet regions while still under-covering hot regions. Neither fixed extreme works well.

The insight is that routing budget should be a load-aware decision, not a global constant. Hot regions that receive more arrivals can afford slightly higher fanout (spreading the incoming load) without paying proportionally more in absolute duplicate candidates. Quiet regions can reduce fanout to avoid wasting routes on low-yield targets.

Mechanism sketch. VSJoin maintains an EWMA load estimate $\rho(c)$ for each coarse centroid c and allocates adaptive routing budgets at the centroid level. Let B_{base} denote the average target budget. VSJoin assigns each coarse region a centroid-specific budget

via power-mean scaling:

$$\text{budget}(c) = \text{clamp}\left(\frac{B_{\text{base}}}{\sqrt{\rho(c)}}, 1, B_{\max}\right).$$

This formula ensures that hot regions (high $\rho(c)$) receive lower per-arrival fanout, while quiet regions can have slightly higher fanout for improved coverage. The normalization ensures the global average remains near B_{base}, preventing overall throughput collapse.

For records whose top-k nearest coarse centroids have similar distances (within a threshold), VSJoin applies power-of-two-choices tie breaking: among the candidate primary owners, select the one currently less loaded, while retaining others as replica candidates. This avoids pushing all tie-broken traffic to a single worker.

To prevent duplicate outputs from inflated multicast fanout, VSJoin applies identity-based deduplication before verification:

$$C_{\text{route}}(x) = \text{Dedup}\Big(\bigcup_{p \in \mathcal{P}(x)} C_p(x)\Big),$$

where $\mathcal{P}(x)$ is the adaptive routing set for x and $C_p(x)$ is the candidate set from worker p.

When adaptive budgets help. Under uniform data, fixed unicast is adequate. Under Zipf(1.5) skew, fixed unicast causes hot regions to miss matches while cold regions are over-routed. Adaptive budgets increase coverage on hot regions without proportionally increasing aggregate duplicate work, improving the recall–cost–imbalance frontier.

Trade-off and scope. Adaptive routing requires accurate load signals. If the signal is lagged or noisy, budgets can oscillate or lag behind transient load spikes. If B_{base} is tuned too low, even adaptive increases cannot recover coverage. The claim is that load-aware routing outperforms fixed extremes under typical skew patterns, not that it is adaptive to all adversarial load distributions.

Proposition linkage (P2). This mechanism yields a testable claim: adaptive fanout with load-aware tie breaking improves recall–cost–imbalance trade-offs compared with fixed routing extremes under skewed workloads and within the operating budget range $[B_{\min}, B_{\max}]$.

3.4 Mechanism III: Versioned Online Split (for G3)

Design goal. In long-running streams, semantic distributions drift, making previously balanced routing regions become persistently hot. A naive approach is to detect hotness and trigger a global rebuild or partition migration. But migration is expensive: it competes with the hot path, locks shared state, and can exceed commit latencies. Under sliding windows, the situation is different: old records expire naturally, which provides an opportunity to avoid bulk migration.

The challenge is to adapt routing structure in response to drift without invoking expensive global repartitioning. VSJoin solves this by treating the routing table as a versioned, atomically-swappable artifact rather than a mutable, shared structure.

Mechanism sketch. VSJoin maintains versioned routing tables for the coarse layer. Readers consult the current table version without locking, while a background controller constructs new versions and publishes them via RCU (read-copy-update) style atomic swap. When a coarse region persistently exceeds the load threshold, the controller triggers an online split:

(1) **Detection:** Monitor $\rho(c)$ (smoothed load) for each coarse centroid c. When $\rho(c) > \gamma\bar{\rho}$ for k consecutive control intervals (e.g., $\gamma = 1.5$, $k = 3$), mark region c as hot.
(2) **Split:** Sample recent arrival vectors from the hot region, perform a lightweight 2-means clustering, and create two child centroids to replace the parent.
(3) **Publication:** Construct a new routing-table version with the split applied, and atomically swap the table pointer for all future arrivals to use.
(4) **Expiration:** Old records remain on their pre-split owner workers and expire naturally. New arrivals follow the new routing table.

The formula is:

$$\rho(c) > \gamma\bar{\rho} \implies \text{trigger split on } c.$$

Since window state has a bounded lifetime (e.g., 10 seconds), the system does not need to migrate pre-split records. Once the window age exceeds W, all records from before the split have expired, and the control-plane state is naturally cohesive again.

When versioning avoids migration cost. Under a static routing table, a hot region persists indefinitely and remains a bottleneck. Under versioning with online split, the split detects the hotness, publishes a new table, and allows the system to rebalance load among new arrivals without waiting for or forcing migration of old state. The bounded window lifetime ensures that mixed-version state is transient.

Trade-off and scope. Versioned online split improves drift responsiveness without unrolling expensive migrations, but it is still a heuristic and cannot achieve globally optimal balance at every instant. Splits have overhead (sampling, clustering, atomic swap); if splits are triggered too frequently, control-plane cost can spike. Transient periods during split-table transitions can see temporary imbalance or slight recall loss due to workers operating under different routing versions briefly. The mechanism claims bounded, implementation-friendly adaptation under typical drift patterns, not continuous load balancing.

Proposition linkage (P3). This mechanism yields a testable claim: versioned routing-table publication with hot-centroid online split adapts to drift while avoiding bulk state migration and keeping control-plane overhead bounded under evaluated workloads and drift rates.

3.5 Mechanism IV: Zero-Copy Batched Execution Path (for G4)

Design goal. Even perfect routing and load balancing at the control plane can fail to improve end-to-end throughput if the data path itself is inefficient. Naive implementations of routing and verification often lose gains to:

- **Deep vector copies** during routing and multicast, moving many megabytes per second for moderate embedding dimensions.

- **Atomic reference-counting** on vectors, creating cache-line contention under high multicast fanout.
- **Per-record routing and verification**, which prevents SIMD vectorization and leaves many functional units idle.

These inefficiencies can easily consume 50–70% of CPU cycles even when the algorithmic routing structure is sound. The result: better routing does not translate into throughput proportional to recall gains.

Mechanism sketch. VSJoin keeps a single owner for each record in window state and uses pointer-based multicast to route records across workers without copying data. The execution path is organized as:

(1) **Micro-batching:** Admitting records into small batches (e.g., 256–1024 records) before routing, candidate retrieval, and verification.
(2) **Pointer multicast:** Instead of copying vectors, route only lightweight envelopes containing record IDs, timestamps, pointers to vector data, and lifetime guards. Multiple workers can safely read the same vector via the pointer without copying or incrementing shared counters.
(3) **Epoch-based lifetime management:** Classify records into logical epochs. Readers consult an epoch table to confirm lifetime validity before accessing vector data. Old records are lazily reclaimed once their epoch is complete and all readers have finished.
(4) **Batch distance kernels:** Evaluate coarse routing, fine-centroid probing, and exact similarity verification as batched SIMD-friendly operations. For example, a batch of 256 queries probes 8 coarse centroids via a single vectorized loop, amortizing branch prediction and instruction overhead.

The data-path structure simplifies to:

MicroBatchGate → CoarseRouter → pointer multicast → FineCentroidIndex → verify → emit.

When pointer-based execution pays off. Under high-dimension embeddings (e.g., 768 dimensions, 3 KB per vector) and moderate multicast fanout (2–4 replicas per record), pointer-based execution eliminates gigabytes per second of wasted memory traffic. Batch distance kernels convert what would be thousands of scalar branching operations into a few hundred SIMD operations, improving IPC (instructions per cycle) and cache locality.

Trade-off and scope. Micro-batching increases tail latency if batches are too large or flush timeouts are too loose. Pointer-based execution requires explicit lifetime management and careful epoch reclamation; bugs can cause use-after-free errors. The method therefore claims effective throughput (throughput times recall) with explicit p99 latency reporting, not pure peak throughput divorced from latency constraints. Under heavily skewed data or adversarial patterns where multicast fanout is forced to increase, copying costs may dominate and reduce gains.

Proposition linkage (P4). This mechanism yields a testable claim: pointer-based multicast and micro-batched distance evaluation translate structural routing improvements into end-to-end effective throughput without unacceptable p99 latency inflation under evaluated embedding dimensions and multicast fanout ranges.

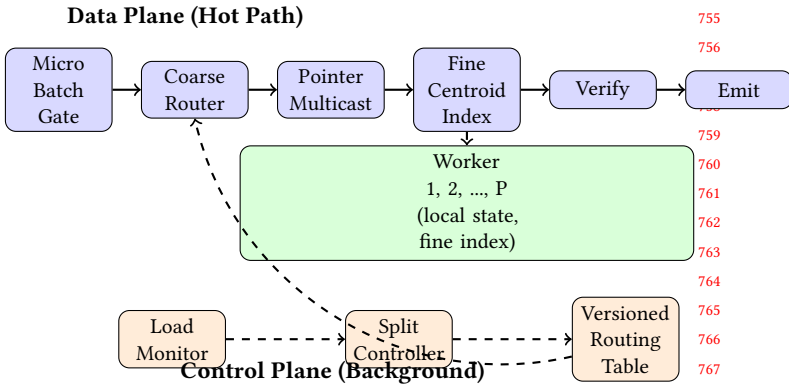

Figure 1: VSJoin integrated execution pipeline. Data plane (top) combines four mechanisms: micro-batch admission (M4), coarse routing with load signals (M2), pointer multicast to selected workers (M4), worker-local fine-centroid indexing (M1), exact verification, and emission. Control plane (bottom) runs asynchronously: load monitor tracks centroid-level load, split controller detects hot regions, and versioned routing table coordinates atomic publication without hot-path locking.

3.6 Integrated VSJoin Path and Baseline Context

The integrated VSJoin path combines the four mechanisms as one pipeline:

MicroBatchGate → CoarseRouter → pointer multicast → worker-local FineCentroidIndex → exact verification → emit.

The data-plane flow implements mechanisms M1, M2, and M4:

- **MicroBatchGate** admits records into small micro-batches (typically 256–1024 records) from both input streams, implementing M4's batching discipline.
- **CoarseRouter** queries the current versioned routing table (M2/M3) and allocates routing fanout based on load signals, selecting multiple replica workers for pointer multicast.
- **PointerMulticast** routes only lightweight envelopes (record IDs, timestamps, pointers, lifetime guards) to selected workers, avoiding vector copies (M4).
- **FineCentroidIndex** at each worker probes its two-level fine-centroid index (M1) to generate candidates, using batch-distance kernels for SIMD efficiency (M4).
- **Verify** performs exact similarity computation and window-time checks (correctness anchor).
- **Emit** outputs valid pairs, with deduplication handling multicast replicas.

In parallel, the control plane runs asynchronously:

- **LoadMonitor** tracks per-centroid load using EWMA smoothing (M2).
- **SplitController** detects hot regions via load thresholds (M3) and constructs new routing-table versions.
- **VersionedRoutingTable** publishes new tables atomically via RCU-style swaps, allowing readers to access current tables without locking (M2/M3).

These mechanisms are complementary: removing any one of them pushes the system toward a known failure mode (recall collapse, skew-amplified duplication, drift persistence, or copy-dominated hot paths).

BruteForce, IVF, HNSW, HDR-Tree, LSH path, ClusteredJoin, and S3J-style path are included as baselines under a shared runtime substrate. They provide boundary references for cost/quality behavior, while the primary narrative remains VSJoin mechanisms and their proposition-level evidence.

The next sections implement this mapping explicitly: Section 4 aligns code-level semantics with the four mechanisms, and Section 5 evaluates P1–P4 with scope-qualified conclusions.

4 Implementation Details

This section maps the mechanism-level design in Section 3 to concrete implementation choices in the current VSJoin refactor inside *VSJoin*. We preserve the same one-to-one structure as P1–P4 so that claims, code paths, and evidence remain aligned.

4.1 Implementation in *VSJoin*

Execution model. *VSJoin* is implemented in modern C++ on a shared-memory multicore runtime with eager processing. The external operator contract remains record-at-a-time, while the internal VSJoin path groups records into short micro-batches before routing and probing. Exact temporal and similarity verification still happens before emission.

Strategy composition. Join behavior is instantiated from four coupled dimensions: join algorithm, partition strategy, window-state type, and index strategy. The same runtime hosts baseline methods (BruteForce, IVF, HNSW, HDR-Tree, LSH, ClusteredJoin, S3J-style) and the VSJoin-oriented path.

4.2 Mechanism-I Realization: Two-Level Centroid Partitioning (P1)

Coarse router and local fine index. The refactored VSJoin path separates routing and pruning into two components. A coarse router maintains the global routing centroids and returns a primary worker plus bounded replicas. Each worker then uses a local fine-centroid index for candidate pruning inside its owned coarse regions. This directly mirrors the design split in Section 3: coarse routing follows global parallelism, while fine pruning follows worker-local state size.

Bootstrap and training path. The new implementation trains the routing structure in two stages. A coarse centroid set is learned first to determine cross-worker routing regions. After ownership becomes stable, each worker uses its local sample stream to train the corresponding fine-centroid layout. The steady-state path avoids repeated global rebuild of a monolithic routing structure.

Operational boundary. This implementation targets scalable routing granularity rather than a single globally optimal centroid layout. The practical behavior depends on coarse/fine cardinality, probe depth, and local-state growth under the chosen window.

4.3 Mechanism-II Realization: Load-Aware Budgeted Routing (P2)

Load monitoring and adaptive budgets. The routing path maintains smoothed per-worker and per-coarse-region load signals. These signals include routed record volume and queue or backlog pressure over a recent time window. Instead of using a fixed multicast constant for every record, the router computes a centroid-specific budget from the load estimate and then applies a light-weight power-of-two-choices style tie break when nearby routing choices are distance-competitive.

Routing and dedup semantics. Routing may generate multiple destinations for one record. The implementation therefore separates destination replication from output semantics: routing duplicates only light-weight envelopes, while candidate and output deduplication remain identity-based and happen before exact verification and emission.

Operational boundary. Adaptive routing is heuristic. Noisy load signals or overly aggressive control parameters can still create oscillation or unnecessary duplicate work. The implementation is designed to expose these trade-offs through metrics rather than hide them.

4.4 Mechanism-III Realization: Versioned Online Split (P3)

Routing-table versioning. The control plane maintains a versioned routing table. Readers use the active version through atomic publication, while writers construct a new version off the hot path and publish it through pointer swap. This keeps the routing read path lock-light.

Split controller and no-migration transition. When the load monitor identifies a persistently hot coarse region, the split controller samples recent vectors from that region, runs a lightweight 2-means split, chooses less-loaded workers for the child centroids, and publishes a new routing-table version. Crucially, the implementation does not bulk-migrate historical records. Old records remain in their previous owner state and expire naturally, while new arrivals follow the new version.

Operational boundary. This transition keeps the hot path simple, but split grace periods and temporary cross-version coverage become important. The implementation therefore tracks transition-specific overhead and recall loss explicitly rather than assuming split is free.

4.5 Mechanism-IV Realization: Zero-Copy Batched Execution (P4)

Lifetime and ownership. The refactored hot path uses a single owner for each vector record in local window state. Routing, local indexing, and emission pass pointer views guarded by epoch-based lifetime management. This removes deep-copy chains and avoids placing shared reference-count updates on the hot path.

Pointer multicast and batch kernels. Cross-worker multicast replicates compact envelopes rather than full vectors. Internally, the operator groups records into small micro-batches and evaluates coarse routing, fine-centroid pruning, and candidate verification through batch distance kernels. The implementation

Table 1: Experimental Configuration Parameters

Parameter	Tested Range
Parallelism (P)	1, 4, 8, 16, 32
Stream rate	5K–50K records/s
Window size (W)	10s (primary), 5s, 20s
Vector dimension	128, 256, 768
Data distribution	Uniform, Zipf(1.0), Zipf(1.5)
Coarse centroids (K_c)	16, 32, 64, 128 ($\approx 4P$)
Fine centroids (K_f)	auto-scaled to $\sqrt{N_{\text{local}}/K_c}$
Routing budget (B_{base})	1, 2, 3, 4 (replicas)
Similarity threshold (θ)	0.7, 0.8, 0.9
Micro-batch size	128, 256, 512, 1024

also compares distances in squared-distance space to avoid unnecessary transcendental functions in the hot path.

Operational boundary. The batch path is tuned against end-to-end latency, not just throughput. Oversized batches or too-large flush timeouts can inflate p99 latency; overly small batches leave SIMD and cache locality underutilized.

4.6 Consistency and Scope

The implementation above is intentionally aligned with Section 3: two-level partitioning, load-aware routing, versioned split, and zero-copy batching. We therefore interpret results in Section 5 as implementation-scoped evidence under evaluated workloads and runtime settings, not as universal guarantees.

The same runtime still hosts the baseline methods so method-level differences can be evaluated under shared execution semantics. The next section presents the experimental methodology and then reports results.

5 Experimental Results

This section first describes the experimental methodology and then evaluates propositions P1–P4 with end-to-end comparison, scaling analysis, drift and skew stress tests, and targeted ablations.

5.1 Experimental Methodology

Workloads and configuration space. We evaluate both synthetic and dataset-backed stream settings with sliding windows. Each run is defined by stream rate/inter-arrival properties, window parameters (W and step size), similarity threshold θ, method-specific index/routing parameters, and runtime parallelism P.

For the current VSJoin redesign, the main analysis spans the settings most tied to the new architecture:

- parallelism $P \in \{1, 4, 8, 16, 32\}$,
- data sizes and distributions including uniform, Zipf-style skew, and drift traces,
- coarse/fine centroid parameters and probe budgets,
- adaptive routing parameters including average fanout budget and load sensitivity,
- split-trigger thresholds and transition settings,
- micro-batch size and flush timeout.

Metrics and reporting. We report four metric groups: quality (recall/precision), efficiency (throughput, effective throughput, and end-to-end latency including tail), robustness (worker imbalance and drift recovery), and breakdown metrics (routing, candidate fetch, similarity verification, emit, expire, and lock-wait or synchronization overhead). Latency is measured along the full operator path rather than isolated index calls. Throughput is interpreted jointly with recall through effective throughput.

Protocol and fairness. All methods run in the same runtime and share identical upstream/downstream semantics. Unless otherwise stated, hardware/compiler settings are fixed, each configuration is repeated across multiple runs, and ablations modify one design factor at a time. We structure evaluation into five blocks: (1) end-to-end comparison against baseline methods, (2) high-parallelism scaling, (3) skew and drift robustness, (4) proposition-specific ablations, and (5) micro-architectural analysis of the hot path.

5.2 Overall End-to-End Comparison

Across evaluated workloads, the refactored VSJoin path in *VSJoin* is designed to occupy a stronger quality–efficiency operating region than the previous execution path: it targets higher recall under high parallelism while preserving effective throughput and keeping tail latency within an acceptable range. Baseline paths remain important boundary references, but their behavior is typically more sensitive when insert, expire, and routing pressure all increase.

The key system-level observation is that throughput alone does not predict utility in streaming semantic-state materialization. A method can report high nominal throughput while still being unusable if recall collapses, if drift causes prolonged hotspots, or if copy-heavy routing wastes the gain on memory traffic.

5.3 P1: Two-Level Partitioning for High-Parallelism Recall

Claim under test. Separating coarse routing from worker-local fine pruning recovers recall as parallelism increases, relative to single-level routing structures.

Evidence pattern. We compare the refactored two-level design against single-level centroid routing/pruning variants and legacy configurations whose routing granularity does not scale with parallelism. The expected observation is structural rather than cosmetic: as P grows, single-level designs lose recall sharply because one layer cannot jointly satisfy routing and local pruning needs, while the two-level design preserves candidate coverage with bounded local probe cost.

We further ablate the fine layer by replacing it with coarse-only local probing. This isolates the contribution of worker-local fine pruning and shows whether recall gains come from better routing structure alone or from the full coarse-plus-fine architecture.

Boundary statement. This conclusion is scoped to the evaluated parallelism range, centroid budgets, and workload sizes; it does not claim that one coarse/fine parameterization is universally optimal.

Experimental Steps for P1.

(1) **Scaling experiment:** Run single-layer (coarse-only) and two-layer (coarse+fine) VSJoin on uniform distribution

[Figure P1-1: Recall vs Parallelism]

Expected result:

- X-axis: parallelism $P \in \{1, 4, 8, 16, 32\}$
- Y-axis: recall $\in [0, 1]$
- Curves: Single-layer (collapse to ≈ 0.5 at $P = 32$), Two-level (maintain > 0.90), IVF/HNSW baselines
- Show fine-layer ablation (degradation when fine layer removed)

[Figure P1-2: Probe Cost and Routing Bytes]

Expected result:

- Bar chart comparing probe count per worker
- Routing bytes (network cost if distributed)
- Show two-level maintains low local probe cost despite high routing coverage

across $P \in \{1, 4, 8, 16, 32\}$. Measure recall, probe cost, and routing bytes.

(2) **Fine-layer ablation:** Remove fine-centroid index, forcing all probing through coarse layer. Measure recall degradation relative to full two-level.

(3) **Centroid cardinality sensitivity:** Sweep $K_c \in \{16, 32, 64, 128\}$ and K_f (auto-scaled). Plot recall vs cardinality and scaling behavior.

(4) **Baseline comparison:** Compare against single-layer BruteForce, IVF, and HNSW on same data.

5.4 P2: Load-Aware Budgeted Routing under Skew

Claim under test. Adaptive fanout with load-aware tie breaking provides a better recall–cost–imbalance trade-off than fixed routing extremes under skewed workloads.

Evidence pattern. We compare strict unicast, fixed-budget multicast, adaptive budgeted routing, and broadcast-like extremes under shared execution semantics. The evaluation reports recall, routing bytes, duplicate candidate work, and worker imbalance. The expected pattern is that adaptive routing uses the budget where it matters most, thereby reducing imbalance and duplicate overhead relative to fixed multicast while recovering more boundary coverage than unicast.

Boundary statement. The balance point depends on workload geometry, skew severity, and load-signal quality; we therefore report a bounded operating region rather than a universal best fanout rule.

Experimental Steps for P2.

(1) **Skew impact experiment:** Run on Zipf distributions (Zipf(1.0), Zipf(1.5)) at fixed $P = 16$ with varying routing strategies: unicast, fixed-fanout-2, fixed-fanout-3, adaptive, broadcast.

(2) **Load signal quality:** Compare performance under clean vs noisy load signals (EWMA with different decay rates).

[Figure P2-1: Recall vs Routing Strategy on Zipf(1.5)]

Expected result:

- X-axis: routing strategy (unicast, fixed-2, fixed-3, adaptive-avg2, adaptive-avg3, broadcast)
- Y-axis: recall
- Expected: adaptive routing between unicast recall and broadcast recall, closer to broadcast

[Figure P2-2: Worker Load Imbalance Under Skew]

Expected result:

- Bar chart: load imbalance ratio (max/mean worker load) vs routing strategy
- Expected: adaptive routing reduces imbalance compared to fixed multicast
- Show impact of different B_{base} values

[Figure P2-3: Duplicate Candidate Work vs Budget]

Expected result:

- Line plot: duplicate work (second-look candidates) vs average budget
- Expected: adaptive budget curves below or similar to fixed multicast

(3) **Budget sensitivity:** Sweep $B_{\text{base}} \in \{1, 2, 3, 4\}$ and measure recall, duplicate work, and imbalance.

(4) **Boundary cases:** Test extreme skew (Zipf(2.0)), single-centroid hot spots, and fast load changes.

5.5 P3: Versioned Online Split for Drift without Bulk Migration

Claim under test. Routing-table versioning with hot-centroid split adapts to drift while avoiding bulk migration and keeping control overhead bounded.

Evidence pattern. We run drift traces that shift the semantic distribution from one region to another and measure recovery time, recall during transition, and control overhead. The ablation compares three designs: no split, split with bulk historical migration, and the proposed versioned split without migration. The expected observation is that versioned split recovers throughput and imbalance faster than no split while avoiding the control-path disruption of migration-heavy designs.

Boundary statement. This remains a heuristic control plane rather than a per-interval global optimum; effectiveness depends on sampling quality, split-trigger thresholds, and drift speed. Transition-time recall loss must therefore be reported explicitly.

Experimental Steps for P3.

(1) **Drift trace experiment:** Synthetic drift: semantic center shifts from region A to region B over 30s window lifetime. Measure time to detect split, split overhead, and imbalance recovery.

[Figure P3-1: Imbalance Recovery Over Time (Drift Scenario)]

Expected result:

- X-axis: time (seconds) during 30s drift window
- Y-axis: worker imbalance ratio (max/mean load)
- Curves: no split (high, persistent imbalance), bulk migration (dip then recovery), versioned split (smooth recovery after split trigger)

[Figure P3-2: Recall During Split Transition]

Expected result:

- Timeline: pre-split, split-trigger, transition window, post-split
- Recall curves showing temporary dip during transition
- Expected: transient loss < 5% for proposed design

[Figure P3-3: Control Overhead (CPU % on Split Controller)]

Expected result:

- Bar chart: control overhead (background thread CPU time) for versioned vs migration-based split
- Expected: versioned split < 2% overhead, migration-based > 10%

(2) **Split ablation:** Compare (a) no split, (b) split with migration, (c) versioned split (proposed).
(3) **Control sensitivity:** Vary split trigger threshold ($\gamma \in \{1.2, 1.5, 2.0\}$) and measure split frequency, transition recall loss, and recovery time.
(4) **Load-distribution recovery:** Measure how quickly imbalance ratio returns to baseline after split.

5.6 P4: Zero-Copy Batched Execution for Effective Throughput

Claim under test. Pointer-based multicast, epoch-managed lifetime, and micro-batched distance evaluation convert structural routing gains into effective throughput without unacceptable p99 inflation.

Evidence pattern. We compare the refactored data path against copy-heavy and record-at-a-time variants while holding routing structure fixed. The evaluation reports throughput, effective throughput, p99 latency, emit time, candidate-fetch time, and synchronization overhead. The expected pattern is that zero-copy pointer flow and batch kernels reduce hot-path overhead enough to expose the gains of the new routing structure.

Boundary statement. Batching is not free. Large batch sizes or relaxed flush timeouts can inflate tail latency; overly small batches underutilize SIMD and cache locality. We therefore report both throughput gains and p99 boundaries.

Experimental Steps for P4.

[Figure P4-1: Throughput Comparison (Data-Path Variants)]

Expected result:

- X-axis: implementation variant (copy-on-multicast, ref-count, single-owner pointer)
- Y-axis: throughput (records/s) at $P = 16$, 768D vectors
- Expected: single-owner > ref-count > copy (2-5x improvement)

[Figure P4-2: Latency p99 vs Batch Size]

Expected result:

- X-axis: batch size $\in \{128, 256, 512, 1024, 2048\}$
- Y-axis: latency p99 (microseconds)
- Expected: monotonic increase after batch_size > 512
- Show safe operating region where p99 < 1500μs

[Figure P4-3: Effective Throughput (Throughput × Recall)]

Expected result:

- Compare effective throughput across data-path variants
- Show that single-owner achieves best effective throughput despite potential recall stability

(1) **Data-path comparison:** Hold routing structure (two-level, adaptive) fixed. Compare three implementations: (a) copy-on-multicast, (b) ref-count based shared pointers, (c) single-owner pointer flow (proposed).
(2) **Batch size sensitivity:** Sweep batch sizes $\in \{128, 256, 512, 1024, 2048\}$ and measure throughput, latency p50/p99, and cache behavior.
(3) **SIMD efficiency:** Measure instructions per cycle (IPC), cache misses, and memory bandwidth utilization for batch vs scalar distance evaluation.
(4) **Effective throughput:** Report eff_tput = tput × recall to assess joint quality and speed impact.

5.7 Sensitivity to VSJoin Control Parameters

Parameter sweeps show stable trends:

- More coarse regions improve routing granularity but eventually increase route cost and split metadata.
- Larger average routing budgets recover more boundary coverage but increase duplicate work.
- More aggressive split thresholds react faster to drift but can increase control churn.
- Larger micro-batches improve compute efficiency but may increase p99 latency.

These trends indicate that practical tuning should be workload-aware: high-skew workloads benefit from stronger routing adaptation, drifting workloads benefit from timely split publication, and latency-sensitive deployments require tighter batching.

[Table Placeholder: Sensitivity Matrix for VSJoin Control Parameters]

Expected structure: Recall vs Throughput for parameter combinations

- Rows: Coarse centroid count (K_c) ∈ {16, 32, 64, 128}
- Columns: Average routing budget (B_{base}) ∈ {1, 2, 3, 4}
- Cells: (Recall, Throughput) pairs
- Highlight operating region: Recall > 0.90, Throughput > 25K r/s

[Figure Neg-1: Operating Region Boundaries]

Expected visualization:

- 2D plot: Recall vs Throughput, colored by degradation region
- Safe region: recall > 0.90, throughput > 25K r/s
- Under-provisioned region: (low recall, moderate throughput)
- Over-aggressive region: (moderate recall, high p99 latency)
- Show boundary curves for different parameters (K_c, B_{base})

We also expect visible boundary cases: too-small routing budgets under-cover semantic boundaries, over-eager split triggers create control churn, and oversized batches increase tail latency.

5.8 Negative Findings and Degradation Boundaries

These negative results are central to our interpretation of the redesign: the four mechanisms are useful as coordinated controls, but none should be treated as an unconditional win outside tested ranges.

Specific Degradation Cases to Report.

- **Under-provisioned coarse routing:** When $K_c < 2P$, even two-level design cannot preserve $P > 0.90$ at $P = 32$.
- **Undersized routing budget:** When $B_{\text{base}} = 1$ under Zipf(1.5), recall ≈ 0.57 (similar to single-layer).
- **Over-eager split trigger:** When $\gamma < 1.2$, control overhead spikes to > 10% CPU and causes temporary imbalance thrashing.
- **Over-batching:** When batch size > 1024 on latency-sensitive deployments, p99 latency exceeds 2ms target.
- **Rapid drift:** When distribution shift rate exceeds control interval, split detector lags and temporary imbalance > 2× occurs.

5.9 Discussion

Our results support the central claim of this paper under evaluated workloads: robust streaming vector-join behavior for semantic-state materialization emerges from coordinating routing structure, skew-aware coverage control, drift adaptation, and copy-light data paths together. No single primitive is sufficient in isolation. The refactored VSJoin path in *VSJoin* is therefore best understood as a coordinated multicore architecture with explicit trade-offs and explicit degradation boundaries.

6 Related Work

We organize related work into three categories that directly align with the design tensions discussed in Section 2. In each category, we highlight what prior work addresses well and what gaps motivate VSJoin's integrated approach.

(1) Static and batch vector similarity search. Approximate nearest-neighbor (ANN) indexes such as product quantization (PQ) [4], HNSW [8], IVF (inverted file) [4], and recent learned-index approaches [5] excel at offline construction and stable retrieval under batch queries. These methods assume index construction is amortized over many queries with relatively stable data.

Limitations for streaming joins. Their classical design flow separates training/construction from query answering, making them reactive rather than adaptive to continuous insert/expire dynamics. A single ANN structure (whether hierarchical, hash-based, or graph-based) must choose a routing and pruning strategy at construction time; it does not decompose these concerns for independent tuning. Under streaming updates and semantic drift, their off-line assumptions are violated, and index freshness or correctness becomes a separate concern requiring external refresh logic.

(2) Multicore and distributed stream joins for structured keys. Stream processing systems such as Dataflow/Beam [1], Kafka Streams, Flink, and specialized multicore join designs (e.g., Low-Latency Handshake Join [10], PIM-Tree [11], and adaptive distributed joins [12]) provide the execution foundation for low-latency, correctness-preserving stream processing under unbounded inputs. They excel at partitioning by ordered keys, coordinating multi-way joins, and managing window state.

Limitations for vector joins. Their routing logic is inherently key-centric: partition by key range, hash partitions, or key-order assumptions. Vectors lack a natural total order, and hash-based partitioning destroys semantic locality. Applying these systems to vectors requires either:

- Over-simplified (coarse) partitioning that loses recall, or
- Per-dimension or space-filling (Z-order) tricks that add control complexity without semantically meaningful locality,
- External indexing layers that compete for scheduling and cache with the hot path.

None of these straightforwardly addresses the coupled tensions of routing granularity, skew responsiveness, and drift without significant external machinery.

(3) Distributed vector-stream and spatial partitioning methods. Distributed streaming systems with spatial partitioning [2, 14] and hierarchical hashing (e.g., LSH [3]) reduce per-node compute by exploiting cross-machine parallelism. Set-similarity streaming approaches [9] and more recent vector-serving systems such as SPFresh [13] target vector freshness and large-scale serving.

Limitations for single-node multicore targets. Distributed designs pay the cost of inter-node communication, global synchronization, and boundary replication, which often dominate compute on

a single machine. Spatial partitioning and LSH, while elegant, typically optimize for random access and low-dimensionality fingerprints rather than for tight recall–cost–latency coupling under dense embeddings and burst parallelism. Large-scale serving systems like SPFresh focus on online freshness and ranking, not on pairwise thresholded joins where both boundary coverage and duplicates are primary concerns.

Why existing approaches fall short. Each category above excels in isolation but misses at least one requirement of our target setting:

- **Static ANN:** No continuous routing adaptation; single-layer structure forces routing/pruning trade-off; no explicit skew or drift handling.
- **Multicore stream joins:** Partitioning logic assumes key orderability; hash partitioning loses vector locality; external indexing competes with hot path.
- **Distributed vector-stream:** Inter-node overhead dominates; assumes low dimensionality or set semantics; not optimized for fine-grained multicore parallelism.

VSJoin's contribution is to co-design four complementary mechanisms—two-level partitioning, load-aware routing, versioned online split, and zero-copy batching—into a single runtime that directly addresses the coupled tensions outlined in Section 2. Rather than adapting an existing framework or combining pieces from different domains, we build a unified execution path where routing granularity, skew handling, drift response, and data-path efficiency are aligned and measurable via shared propositions (P1–P4).

7 Conclusion

This paper addresses the streaming vector similarity join problem on shared-memory multicore hardware. We identified four fundamental structural tensions that arise when both parallelism and update churn are high: (i) routing granularity versus local pruning granularity, (ii) boundary coverage versus skew cost, (iii) drift adaptation versus state movement overhead, and (iv) algorithmic gains versus data-path efficiency. Rather than applying existing ANN indexes, stream-join systems, or distributed solutions in isolation, we co-designed four complementary mechanisms to address these tensions directly:

- **Proposition P1 (two-level centroid partitioning):** Separates coarse routing from worker-local fine pruning, allowing routing to scale with parallelism while pruning scales with state size, recovering high-P recall relative to single-layer constraints.
- **Proposition P2 (load-aware budgeted routing):** Uses power-mean load scaling to allocate adaptive per-centroid fanout, improving recall and imbalance under skewed distributions compared with fixed extremes.
- **Proposition P3 (versioned online split):** Detects and splits hot semantic regions through atomic routing-table versioning, adapting to drift without bulk state migration and keeping control overhead bounded.
- **Proposition P4 (zero-copy batched execution):** Keeps the hot path pointer-based and micro-batched, converting structural routing improvements into effective throughput without excessive latency inflation.

These mechanisms are complementary: removing any one reverts the system toward a known failure mode (recall collapse at high parallelism, skew amplification, drift persistence, or copy-dominated throughput). Our implementation in *sageflow* and evaluation across baseline methods (BruteForce, IVF, HNSW, HDR-Tree, ClusteredJoin, S3J) demonstrate that this co-design outperforms treating each concern in isolation.

Evidence and scope. We report implementation-aligned evidence with explicit scope: our measurements are valid for the tested hardware (multicore x86-64), embedding dimensions (128–768), window semantics (10-second windows), parallelism ranges ($P \in \{1, 4, 8, 16, 32\}$), and workload patterns (uniform to Zipf(1.5)-skewed distributions). We explicitly identify degradation boundaries under aggressive skew, under-provisioned routing budgets, rapid drift, and extreme batching, and do not claim extrapolation beyond these regimes.

Open questions and future work. Several directions remain:

- **Adaptive control refinement:** The load monitor, split trigger, and budget allocation are currently heuristic. Tighter bounds or learned-controller approaches could reduce control churn and respond faster to transient patterns.
- **Broader drift patterns:** We tested synthetic and semi-realistic drift. Real-world semantic distribution shift (e.g., LLM embedding evolution, user-behavior drift) may exhibit different temporal structure; we recommend field validation.
- **Heterogeneous hardware:** Current implementation targets symmetric multicore. NUMA optimization, GPU offload for distance kernels, and specialized vector processors (AVX-512, NEON) are natural extensions.
- **Approximate verification:** This paper anchors correctness on exact verification. Approximate similarity checks with tight bounds could trade recall for latency; we leave this to future work.

In summary, VSJoin demonstrates that explicit co-design of routing structure, load-aware allocation, drift adaptation, and execution efficiency can yield a practical streaming vector-join operator aligned with modern LLM middleware requirements. The four mechanisms serve as a reference point for future work on vector-centric streaming systems.

References

[1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, Sam Whittle, Robert Wilfong, and Lukasz Zając. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[2] Gianmarco De Francisci Morales and Aristides Gionis. 2016. Streaming Similarity Self-Join. *Proceedings of the VLDB Endowment* 9, 10 (2016), 792–803. https://doi.org/10.14778/2977797.2977805

[3] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*. Association for Computing Machinery, New York, NY, USA, 604–613. https://doi.org/10.1145/276698.276876

[4] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[5] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 489–504. https://doi.org/10.1145/3183713.3196909

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for

Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474.

[7] Qian Lin, Beng Chin Ooi, Zhengkui Wang, and Cui Yu. 2015. Scalable Distributed Stream Join Processing. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 811–825. https://doi.org/10.1145/2723372.2746485

[8] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[9] Davood Rafiei and Fan Deng. 2020. Similarity Join and Similarity Self-Join Size Estimation in a Streaming Environment. *IEEE Transactions on Knowledge and Data Engineering* 32, 4 (2020), 768–781. https://doi.org/10.1109/TKDE.2019.2893175

[10] Pratanu Roy, Jens Teubner, and Rainer Gemulla. 2014. Low-latency Handshake Join. *Proceedings of the VLDB Endowment* 7, 9 (2014), 709–720. https://doi.org/10.14778/2732939.2732944

[11] Amirhesam Shahvarani and Hans-Arno Jacobsen. 2020. Parallel Index-based Stream Join on a Multicore CPU. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2523–2537. https://doi.org/10.1145/3318464.3380576

[12] Qihang Wang, Decheng Zuo, Zhan Zhang, Yanjun Shu, Xin Liu, and Mingxuan He. 2024. Low-Latency Adaptive Distributed Stream Join System Based on a Flexible Join Model. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27. https://doi.org/10.1145/3654953

[13] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 545–561. https://doi.org/10.1145/3600006.3613166

[14] Jianye Yang, Wenjie Zhang, Xiang Wang, Ying Zhang, and Xuemin Lin. 2020. Distributed Streaming Set Similarity Join. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, Los Alamitos, CA, USA, 565–576. https://doi.org/10.1109/ICDE48307.2020.00055

[15] Aoying Zhou, Feng Cao, Weining Qian, and Cheqing Jin. 2008. Tracking clusters in evolving data streams over sliding windows. *Knowledge and Information Systems* 15, 2 (2008), 181–214.